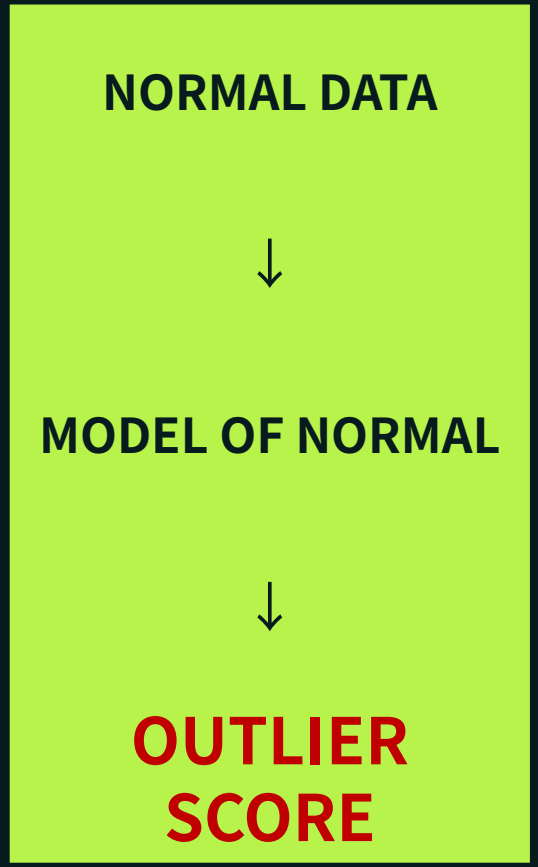
Anomaly Detection
Unsupervised Methods

Finding the odd ones out when nobody has labelled them



Financial Machine Learning · Supplementary 3

Four model families learn what normal looks like

Every method here is unsupervised — no outlier labels are needed

01	**One-Class SVM & SVDD**	**a boundary around normal data** hyperplane · smallest hypersphere
02	**Isolation Forest**	**outliers are easy to isolate** random splits · path length · anomaly score
03	**Local Outlier Factor**	**density relative to neighbours** reachability distance · lrd · LOF ratio
04	**Autoencoders**	**outliers reconstruct badly** normal-only training · error · LSTM

Start with domain sense, then move to multivariate models

Three levels of outlier detection, from simplest to most flexible

LEVEL 1

Domain-based EDA

- Plots and business rules
- Know what is impossible

LEVEL 2

Univariate rules

- Tukey fences (IQR)
- z-score thresholds

LEVEL 3 · THIS DECK

Multivariate models

- MCD · One-Class SVM
- Isolation Forest · LOF
- Autoencoders

One variable at a time misses points that are odd only in combination.

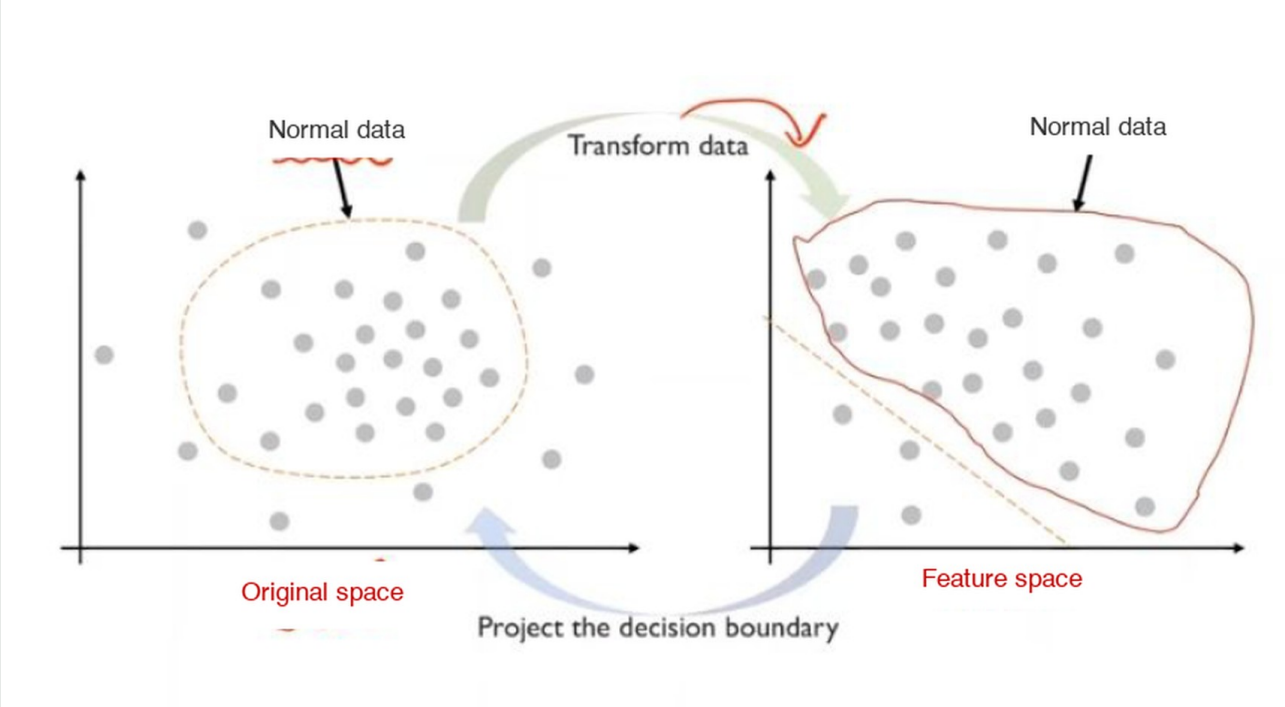
01

One-Class SVM & SVDD

Draw a boundary around the normal data, flag what falls outside

One-Class SVM pushes the normal data away from the origin

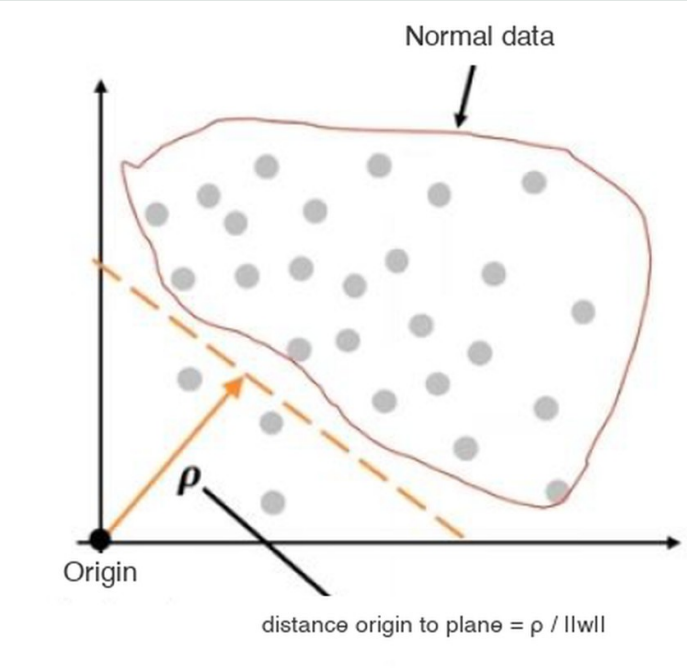
Map the data to a feature space, separate it from the origin, project the boundary back



※ **An SVM trained on one class only: it learns the region of normal observations, not the outliers**

The hyperplane sits as far from the origin as possible

The plane w · ϕ(x) = ρ lies ρ/ ‖ w ‖ from the origin; ν bounds the share of outliers



Objective

$$\min_{w,\,\xi,\,\rho} \ \tfrac{1}{2}\|w\|^2 + \tfrac{1}{\nu n}\sum_{i=1}^{n}\xi_i - \rho$$

Constraints

$$\text{s.t.} \quad w \cdot \phi(x_i) \geq \rho - \xi_i, \qquad \xi_i \geq 0$$

Decision: +1 normal · −1 outlier

$$f(x) = \operatorname{sign}(w \cdot \phi(x) - \rho)$$

Maximise ρ while letting a few points (slack ξ) cross — those are the outliers.

Each term of the objective has one job

Together they maximise the margin between the origin and the normal data

Margin

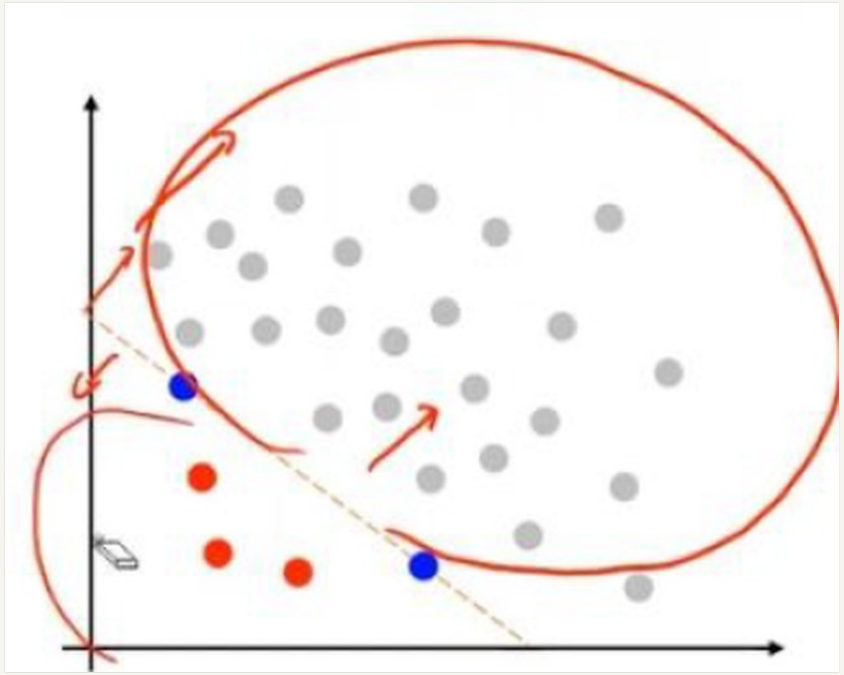
$$\text{distance}(\text{origin},\ \text{hyperplane}) = \frac{\rho}{\|w\|}$$

- $\frac{1}{2}\|w\|^2$ — keep the norm of w small: a wider margin and a simpler, smoother boundary
- $-\rho$ — make ρ large: push the plane away from the origin
- $\frac{1}{\nu n}\sum_i \xi_i$ — **penalty for the points that cross — ν caps their share**

A small norm of w and a large ρ both widen the margin — the same idea as an ordinary SVM.

The multiplier α tells where each observation sits

From the dual problem of the One-Class SVM



Dual constraints

$$\alpha_i = \tfrac{1}{\nu n} - \beta_i, \quad \sum_i \alpha_i = 1, \quad 0 \le \alpha_i \le \tfrac{1}{\nu n}$$

- $\alpha_i = 0$ — inside the boundary — normal
- $0 < \alpha_i < \tfrac{1}{\nu n} \Rightarrow \xi_i = 0$ — on the boundary — support vector
- $\alpha_i = \tfrac{1}{\nu n} \Rightarrow \xi_i > 0$ — **outside — support vector, outlier**

Only the support vectors (α > 0) shape the boundary.

In scikit-learn, predict returns −1 for an outlier

sklearn.svm.OneClassSVM — five one-dimensional points

READING THE OUTPUT

- 0 and 1 are flagged −1
- score_samples: lower = odder
- gamma sets the kernel width

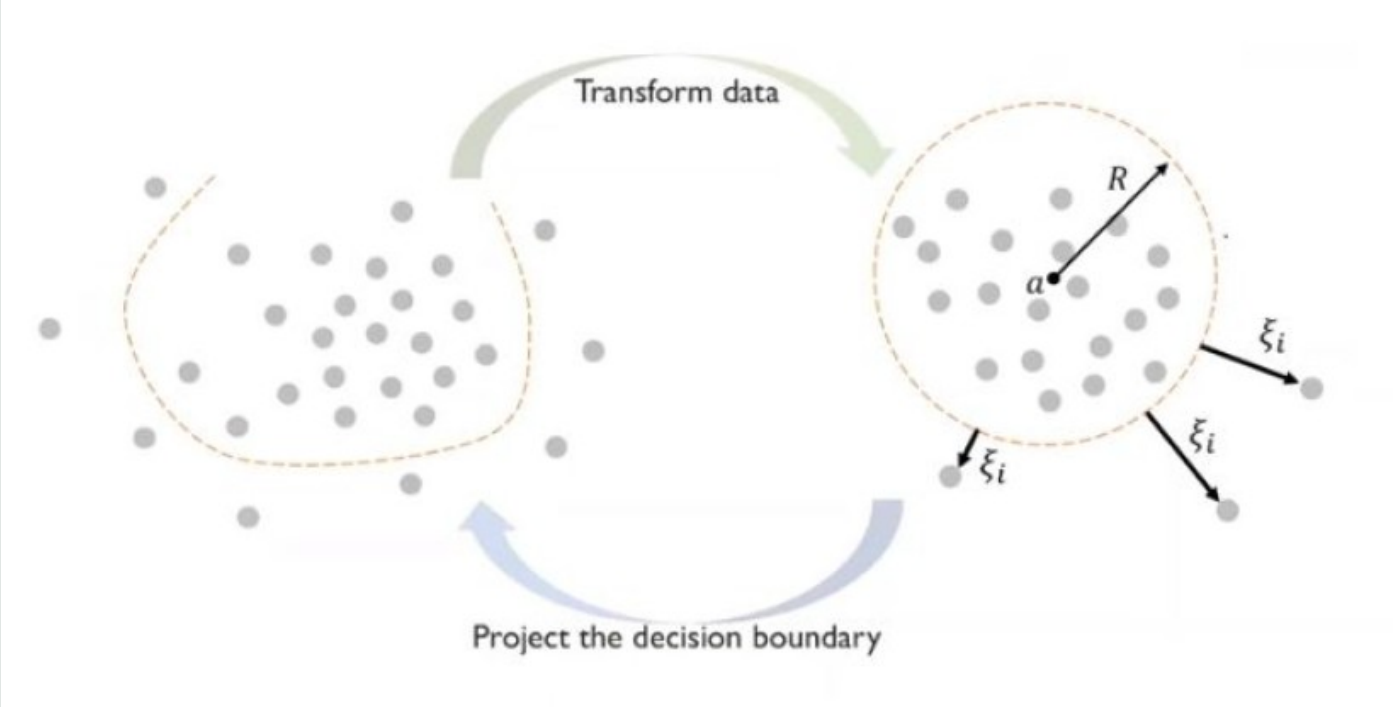
Fit on normal data, then score new data

Examples

```
>>> from sklearn.svm import OneClassSVM
>>> X = [[0], [0.44], [0.45], [0.46], [1]]
>>> clf = OneClassSVM(gamma='auto').fit(X)
>>> clf.predict(X)
array([-1,  1,  1,  1, -1])
>>> clf.score_samples(X)
array([1.7798..., 2.0547..., 2.0556..., 2.0561..., 1.7332...])
```

SVDD wraps the normal data in the smallest hypersphere

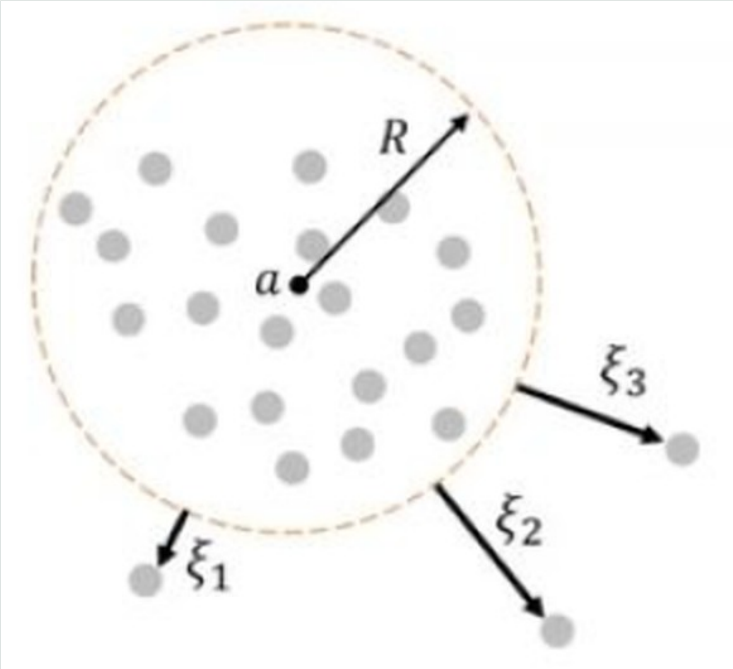
Support Vector Data Description — centre a, radius R, slack ξ for points outside



※ **Same transform-then-project idea as the One-Class SVM; the boundary is a sphere instead of a hyperplane**

SVDD trades a small radius against points left outside

ϕ = kernel mapping · C = regularisation · n = number of normal observations



Objective: shrink the sphere, penalise distances outside

$$\min_{R,\, a,\, \xi} \; R^2 + C\sum_{i=1}^{n}\xi_i$$

Constraints

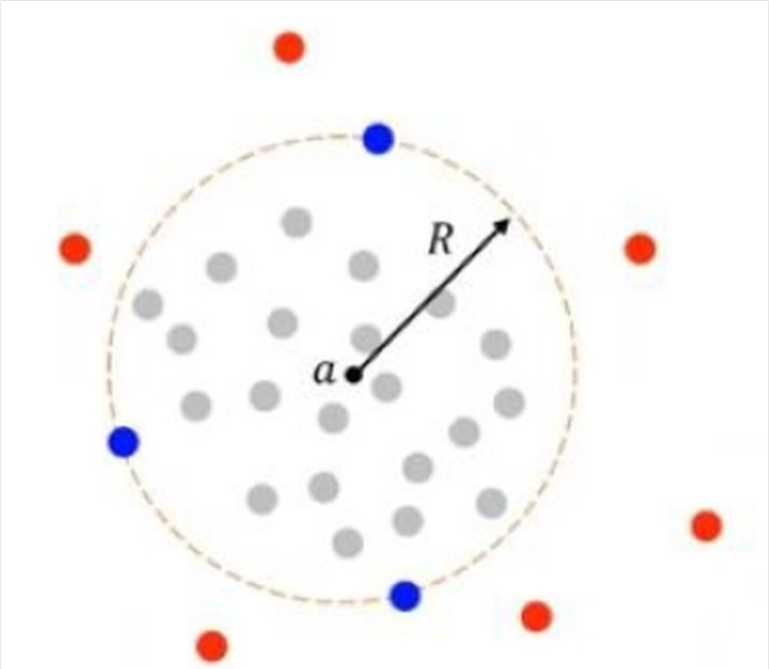
$$\text{s.t.} \quad \|\phi(x_i) - a\|^2 \le R^2 + \xi_i, \qquad \xi_i \ge 0$$

Large C keeps almost every point inside; small C allows a tighter sphere and more outliers.

With an RBF kernel, SVDD and the One-Class SVM give the same boundary: minimising R = maximising ρ.

As in the One-Class SVM, α places each observation

From the dual problem of SVDD



Dual constraints

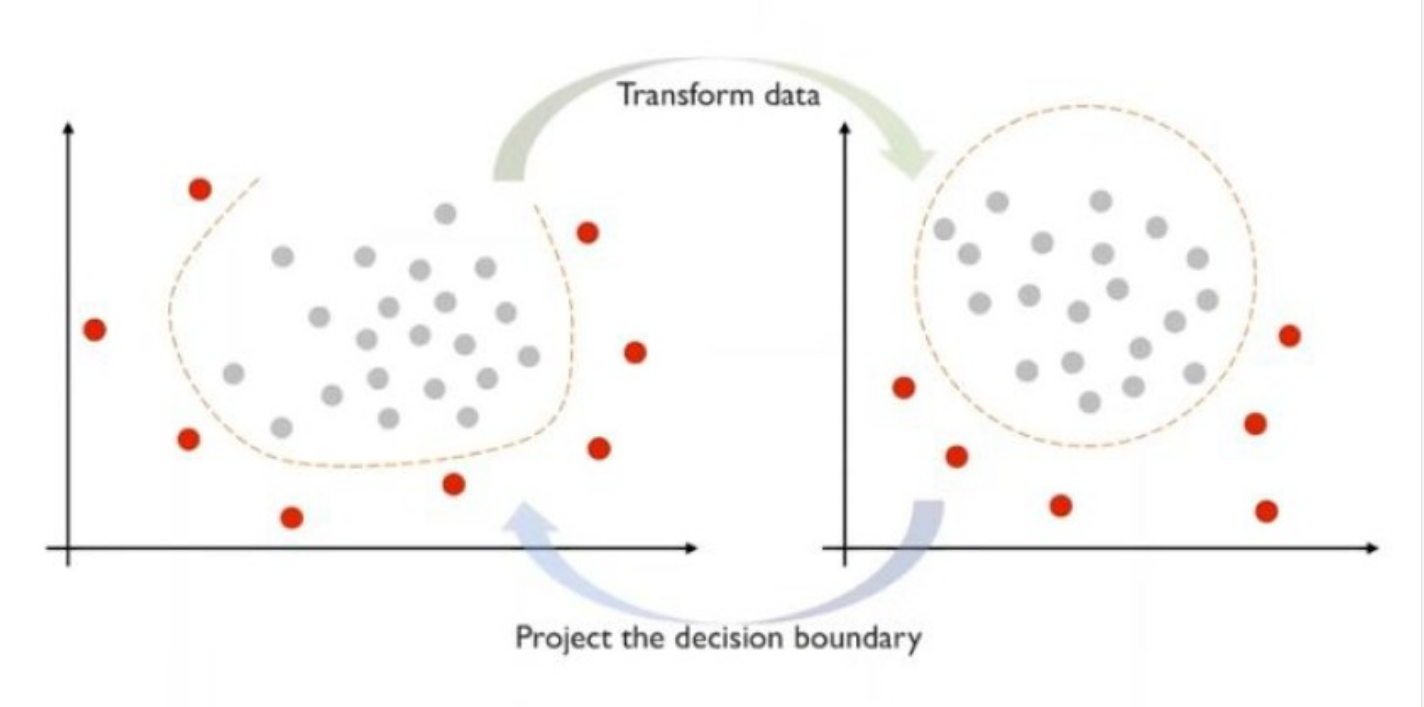
$$C - \alpha_i - \beta_i = 0, \quad \beta_i \xi_i = 0, \quad 0 \le \alpha_i \le C$$

- $\alpha_i = 0$ — inside the sphere — normal
- $0 < \alpha_i < C \Rightarrow \xi_i = 0$ — on the sphere — support vector
- $\alpha_i = C \Rightarrow \xi_i > 0$ — **outside — support vector, outlier**

Points outside the sphere are the outliers.

Observations outside the sphere are judged outliers

Find the hypersphere around normal data in feature space, then project its boundary back



※ **Implementation walk-through: zephyrus1111.tistory.com/471**

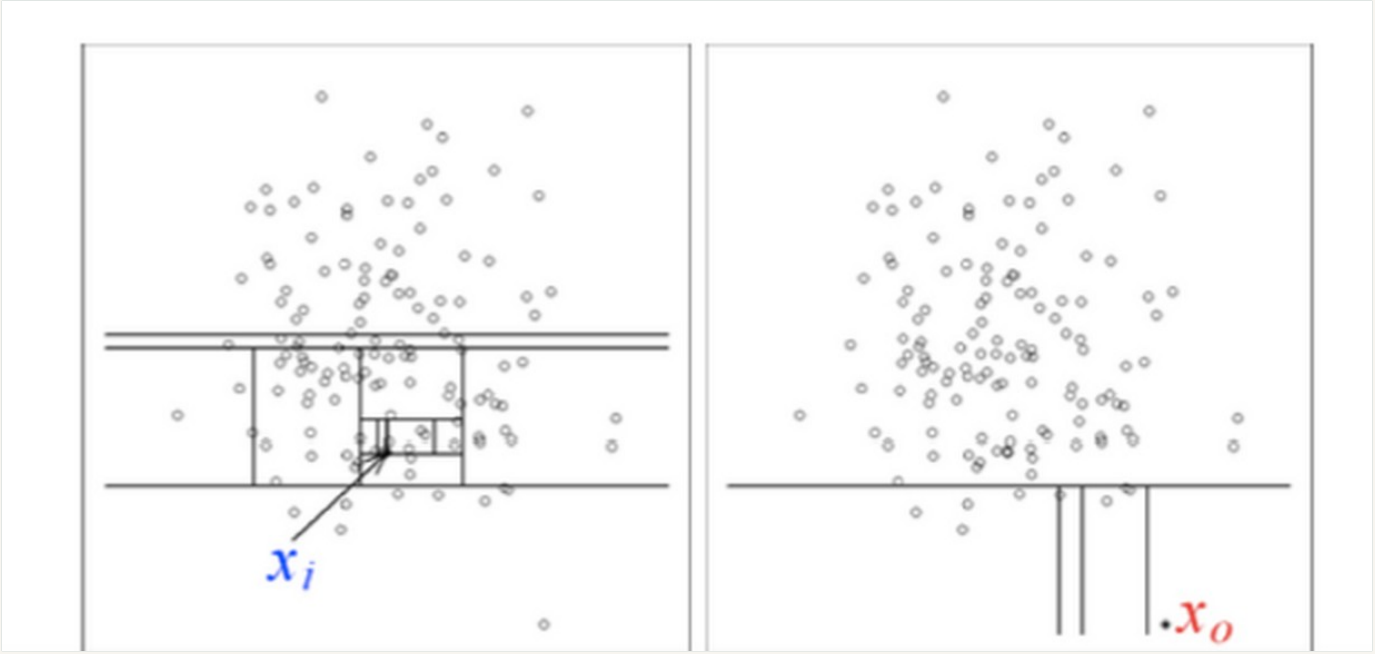
02

Isolation Forest

Outliers are few and different, so they are easy to isolate

A normal point needs many cuts; an outlier needs few

Each iTree splits at random to isolate observations — no labels needed



The point in the crowd takes many splits; the lone point takes only a few.

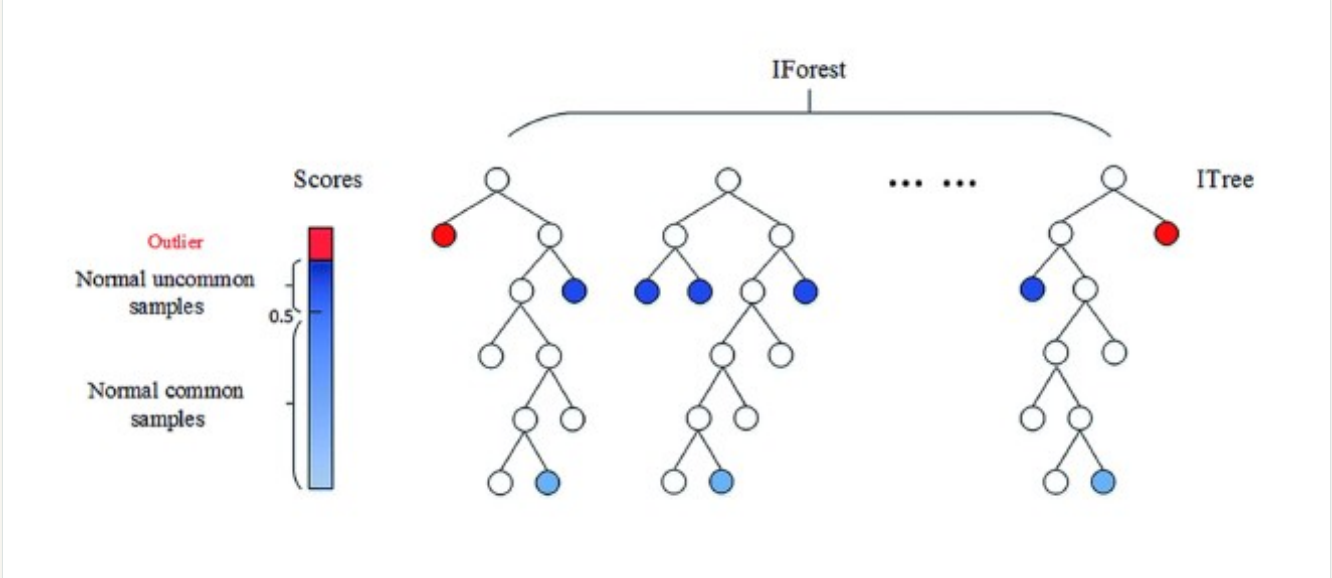
Random splits on random features build each tree

Repeat until every point is isolated or a depth limit is reached

HOW ONE iTREE GROWS

- Pick a feature at random
- Cut uniformly in [min, max]
- Record each path length

Short path = likely outlier



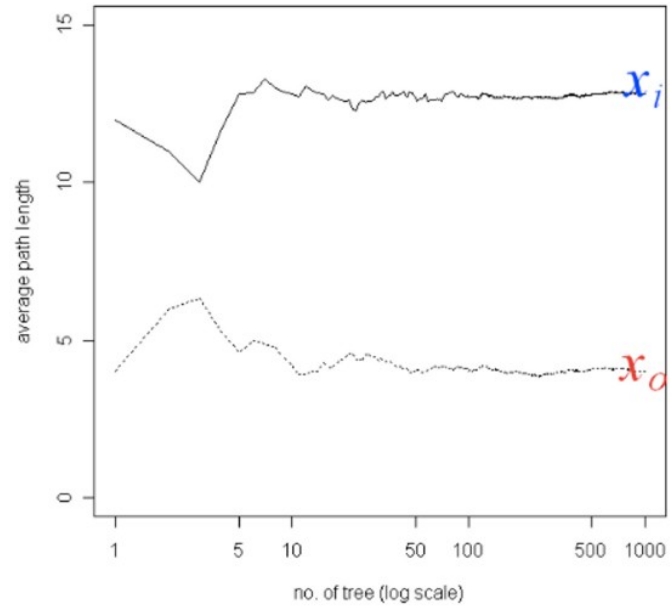
Averaged over many trees, the path lengths settle

Average path length of the crowded and the lone point as the number of trees grows (log scale)

WHAT THE CHART SHOWS

- Crowded point: about 12–13
- Lone point: about 4
- Stable after ~100 trees

The forest separates the outlier robustly



(c) Average path lengths converge

The anomaly score turns path length into a number near 0.5 or 1

Average depth = splits until a point stands alone, averaged over the trees

Anomaly score, in words

$$s = 2^{-\text{average depth} \,/\, \text{typical depth}}$$

Typical depth for n points

$$\text{typical depth } c(n) \approx 2\ln n - 1 \qquad (\approx 10 \text{ for } n = 256)$$

READING THE SCORE

much shorter → s ≈ 1

equal → s = 0.5

longer → s < 0.5

With 256 points the typical depth is about 10: isolated in 4 splits gives s ≈ 0.76, in 10 splits s ≈ 0.5.

In scikit-learn, IsolationForest flags the far value −1

sklearn.ensemble.IsolationForest — fit on four values, predict three new ones

READING THE OUTPUT

- 0.1 and 0 → +1 (normal)
- 90 → −1 (outlier)
- No labels used in fit

−1 = outlier, +1 = inlier

Examples

```
>>> from sklearn.ensemble import IsolationForest
>>> X = [[-1.1], [0.3], [0.5], [100]]
>>> clf = IsolationForest(random_state=0).fit(X)
>>> clf.predict([[0.1], [0], [90]])
array([ 1,  1, -1])
```

03

Local Outlier Factor

Compare a point's density with the density of its neighbours

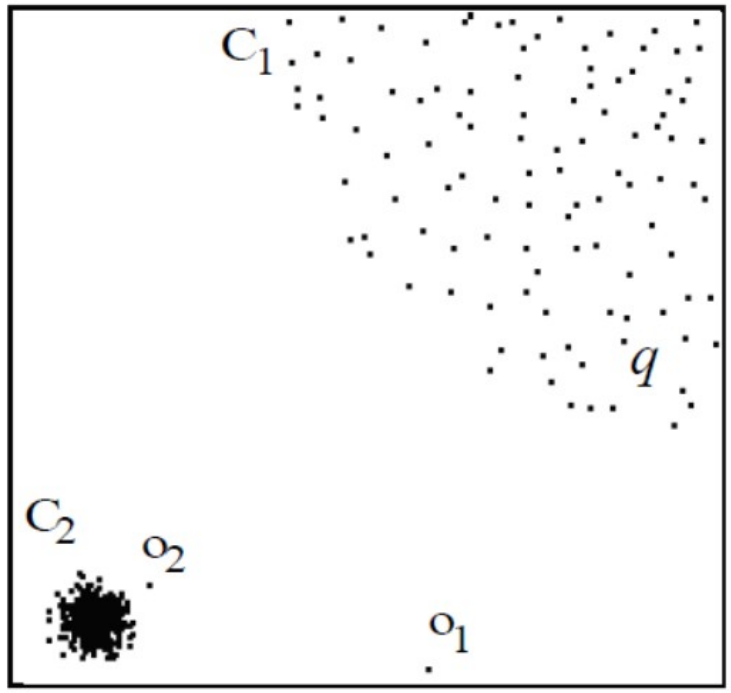
Is O2 an outlier relative to cluster C2?

A sparse cluster C1, a dense cluster C2, and two candidate outliers

THE PROBLEM

- O1 is far from everything
- O2 is close in absolute terms
- …but far for dense C2

A global distance cutoff misses O2



Local density is one over the average reach distance

Three definitions in words — here with k = 3 neighbours

Neighbourhood radius

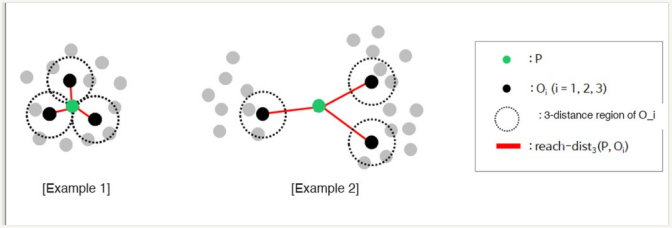
$$\text{radius}(o) = \text{distance from } o \text{ to its } k\text{th nearest neighbour}$$

Reach distance — never shorter than o's own radius

$$\text{reach}(p \to o) = \max\{\text{radius}(o),\ d(p, o)\}$$

Local density

$$\text{density}(p) = \frac{1}{\text{average reach from } p \text{ to its } k \text{ neighbours}}$$



Long reach distances to the neighbours mean low density (Example 2).

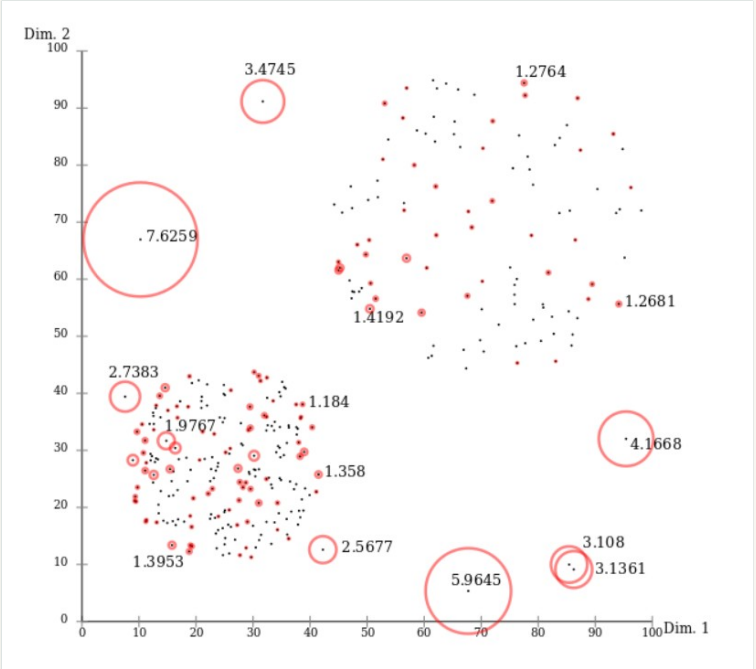
LOF compares the neighbours' density with the point's own

The neighbours' average density divided by the point's own density

Local Outlier Factor, in words

$$\mathrm{LOF}(p) = \frac{\text{average density of the neighbours}}{\text{density of } p}$$

LOF < 1	denser than its neighbours
LOF ≈ 1	similar to its neighbours
LOF ≫ 1	sparser — an outlier



Isolated points score about 2.5 to 7.6; clustered points stay near 1.

In scikit-learn, fit_predict labels the far point −1

sklearn.neighbors.LocalOutlierFactor with n_neighbors = 2

READING THE OUTPUT

- 101.1 is flagged −1
- negative_outlier_factor_ = −LOF
- −73.4 vs about −1

More negative = more outlying

Examples

```
>>> import numpy as np
>>> from sklearn.neighbors import LocalOutlierFactor
>>> X = [[-1.1], [0.2], [101.1], [0.3]]
>>> clf = LocalOutlierFactor(n_neighbors=2)
>>> clf.fit_predict(X)
array([ 1,  1, -1,  1])
>>> clf.negative_outlier_factor_
array([ -0.9821...,  -1.0370..., -73.3697...,  -0.9821...])
```

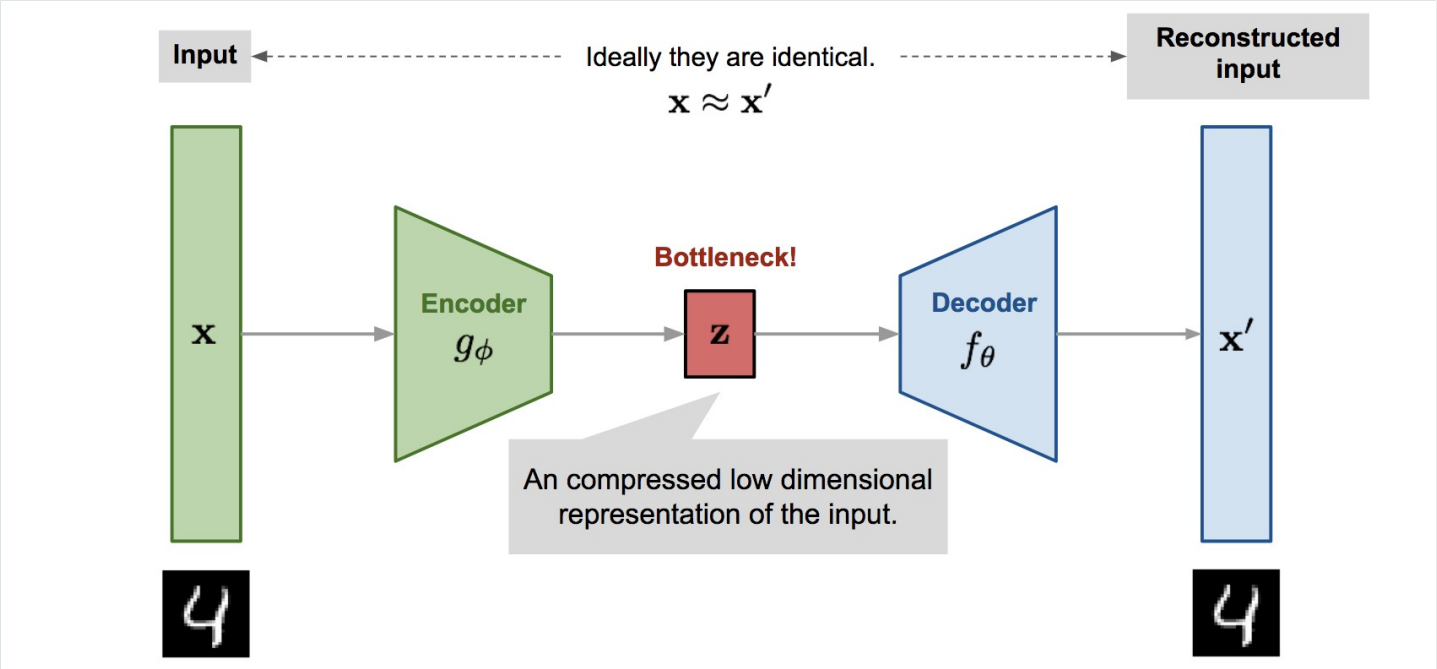
04

Autoencoders

A network trained on normal data reconstructs outliers badly

An autoencoder trained on normal data cannot rebuild an outlier

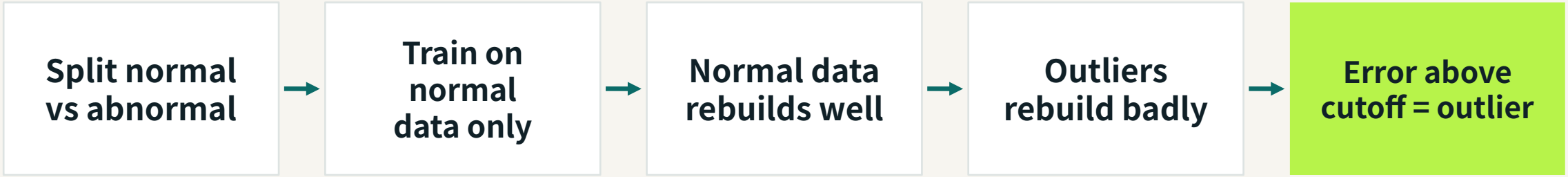
Encoder g_ϕ compresses x to a bottleneck z; decoder f_θ rebuilds x′ ≈ x



Ideally x′ ≈ x — the bottleneck keeps only the main features of normal data.

High reconstruction error marks an outlier

Five steps of autoencoder-based outlier detection



Reconstruction error and cutoff τ

$$e(x) = \|x - f_\theta(g_\phi(x))\|^2 > \tau \Rightarrow \text{outlier}$$

The network never saw outliers in training, so it cannot rebuild them.

Label as outliers the data whose reconstruction error exceeds τ.

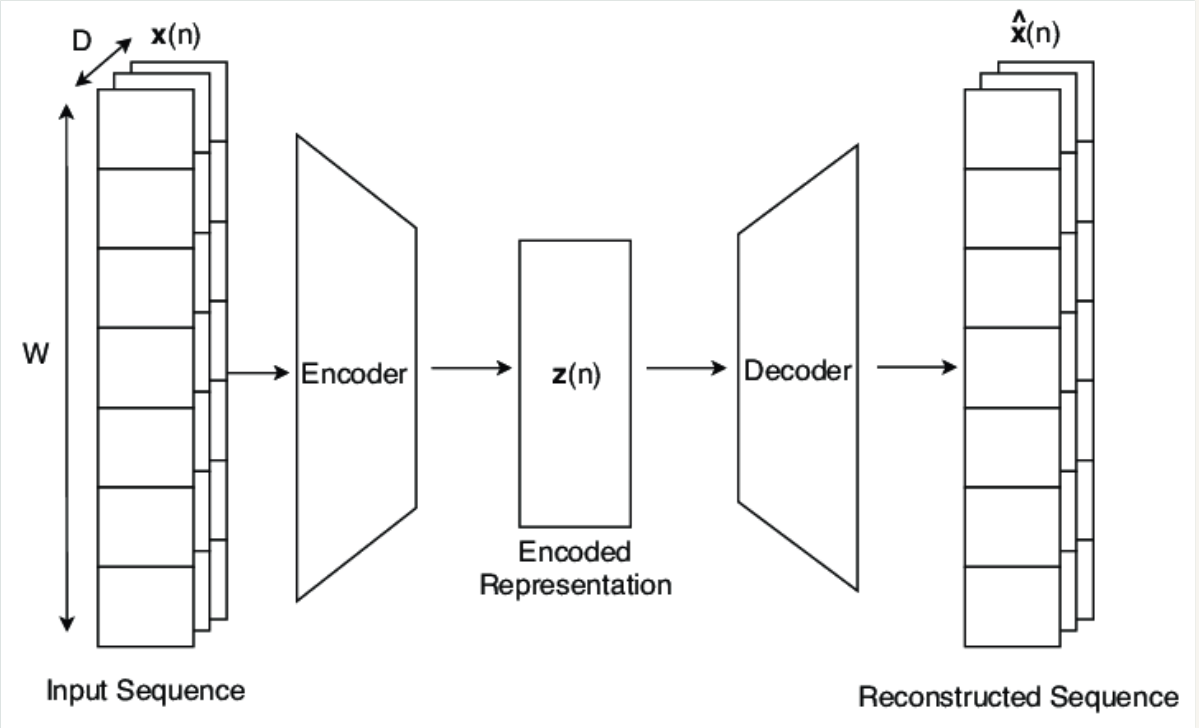
An LSTM autoencoder applies the same idea to sequences

Encode a window of time steps and features, rebuild it, score the error

TIME-SERIES OUTLIERS

- LSTM cells keep memory
- Error per window
- Spikes mark anomalies

Code: cran2367 on GitHub



Each family defines 'normal' in a different way

Pick the one whose idea of normal matches your data

OCSVM · SVDD

A boundary

tune ν or C

ISOLATION FOREST

Easy to isolate

score near 1

LOF

Locally sparse

LOF ≫ 1

AUTOENCODER

Hard to rebuild

error above τ

No labels needed — but always check the flagged points with domain sense.

LEARN WHAT NORMAL LOOKS LIKE.

SCORE HOW FAR EACH POINT STRAYS.

FLAG THE ODD ONES OUT.

Boundary, isolation, density,
reconstruction.